

HTML CSS JS

INFO

Lesinhoud

We geven een overzicht van de basisconcepten van HTML, CSS en JavaScript, inclusief structuur, opmaak en interactiviteit van webpagina's.

▼ Inhoud

- [HTML](#)
- [CSS](#)
 - [Selectors](#)
 - [Eigenschappen](#)
 - [Flexbox](#)
 - [CSS Reset](#)
 - [Media Queries](#)
 - [CSS Custom Properties \(Variabelen\)](#)
- [JavaScript](#)
 - [Variabelen](#)
 - [Conditionele statements](#)
 - [Iteratieve statements](#)
 - [Functies](#)
 - [DOM-manipulatie](#)

We bekijken de "Vanilla"-versies van HTML, CSS en JavaScript, dus zonder frameworks of libraries. Deze basiskennis helpt je straks om met een framework te werken.

HTML

HTML geeft de structuur en inhoud van een webpagina weer. Je kan verschillende elementen gebruiken, zoals `h1`, `h2`, `p`, `a`, `img`, `div`, `span`, ... Ook kan je semantische structuurelementen gebruiken zoals `header`, `footer`, `article`, `section`, `nav`, ... om de inhoud beter te organiseren.

Via attributen voeg je extra informatie toe aan HTML-elementen, zoals `class`, `id`, `src`, `href`, `alt`, ...

Met `emmet abbreviations` genereer je snel HTML in moderne IDE's zoals VSCode, JetBrains WebStorm, ...

`<tab>` duidt op het indrukken van de `tab`-toets om de afkorting uit te breiden naar volledige HTML-code.

- `! <tab>`, genereert de standaard inhoud van een HTML5-document
- `div <tab>`, genereert een `<div></div>`-element
- `div*5 <tab>`, genereert vijf `<div></div>`-elementen
- `div.container>p*3 <tab>`, genereert een `<div>` met de class `container` en drie paragrafen genest in de div
- `div#special>p.highlighted*3 <tab>`, genereert een `<div>` met het id `special` en drie paragrafen met de class `highlighted` genest in de div
- `lorem5 <tab>`, genereert vijf woorden met `lorem ipsum`-tekst
- `lorem50 <tab>`, genereert vijftig woorden met `lorem ipsum`-tekst
- `ul>li*4>{Item $} <tab>`, genereert een ongeordende lijst met vier lijstitems, genummerd van 1 tot 4

- `section*2>h2{Title $}+p>lorem20 <tab>` , genereert twee secties, elk met een titel met als inhoud `Title 1` en `Title 2` , en een paragraaf met twintig woorden `lorem ipsum` -tekst
-

CSS

CSS bepaalt de opmaak en het design van een webpagina. Je kan CSS op verschillende manieren toepassen: inline, internal (in een `<style>` -element) of external (via een `.css` -bestand dat je koppelt via een `<link>` -element).

Selectors

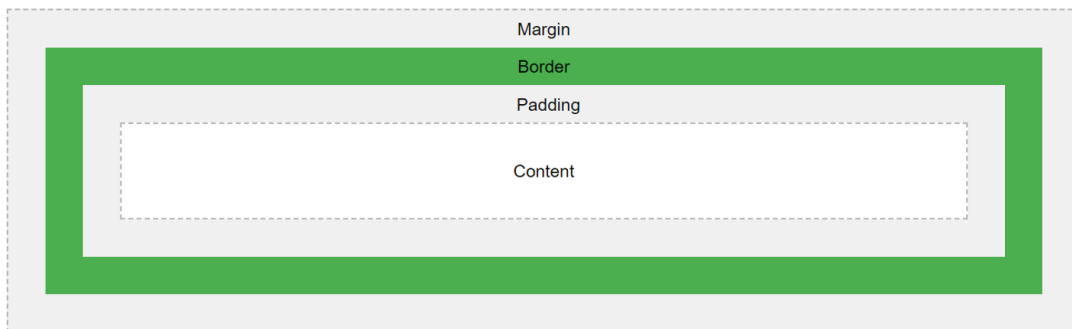
Met `selectors` selecteer je specifieke HTML-elementen en pas je ze aan. Je kan verschillende soorten selectors gebruiken:

- Wildcard selector: selecteert alle elementen, bv. `*`
- Element selectors: selecteren op basis van het HTML-element, bv. `p` , `h1` , `div` , ...
- Contextuele selectors: selecteren op basis van de hiërarchie van elementen, bv. `div p` (alle paragrafen binnen een div), `section > h2` (alle h2's die directe children zijn van een section), ...
- Class selectors: selecteren op basis van de class-attribuut, bv. `.container` , `.highlighted` , ...
- ID selectors: selecteren op basis van het id-attribuut, bv. `#special` , `#header` , ...
- Attribute selectors: selecteren op basis van een attribuut, bv. `[type="text"]` , `[href^="https"]` , ...
- Pseudo-classes: selecteren op basis van een specifieke staat van een element, bv. `:hover` , `:focus` , `:nth-child()` , ...
- Pseudo-elements: selecteren en stylen van een specifiek deel van een element, bv. `::before` , `::after` , `::first-letter` , ...

Eigenschappen

Met CSS stel je verschillende eigenschappen in, zoals `color`, `background-color`, `font-size`, `margin`, `padding`, `border`, `display`, `position`, ...

Via het `box model` bepaal je de witruimte van een element, bestaande uit `content` (de inhoud) en de eigenschappen `padding`, `border` en `margin`.



De `display`-eigenschap is vaak `block` of `inline`. Het is belangrijk om te weten dat `block`-elementen een volledig "blok" innemen en op een nieuwe regel beginnen, terwijl `inline`-elementen enkel de ruimte innemen die ze nodig hebben om hun inhoud te tonen en de elementen kunnen naast elkaar staan. `inline-block` is een combinatie van beide. `inline-block` gedraagt zich als een `inline` element doordat het enkel de ruimte inneemt die het nodig heeft, maar je kan het de eigenschappen geven van een `block` element (`width`, `height`, `padding`, `margin`, ...).

De `position`-eigenschap bepaalt hoe je een element op de pagina plaatst. De meest gebruikte waarden zijn `static` (standaard), `relative` (ten opzichte van de static-positie), `absolute` (ten opzichte van de volledige pagina of ten opzichte van de eerste parent met position relative), `fixed` (ten opzichte van het viewport) en `sticky` (blijft plakken aan de viewport bij scrollen).

Flexbox

Met Flexbox bouw je een flexibele layout. Enkele belangrijke eigenschappen zijn:

- `display: flex;` om een flex-container te maken
- `flex-direction` om de richting van de flex-items te bepalen (`row` , `column` , ...)
- `justify-content` om de uitlijning van de flex-items langs de hoofdas te bepalen (`flex-start` , `center` , `space-between` , ...)
- `align-items` om de uitlijning van de flex-items langs de kruis-as te bepalen (`flex-start` , `center` , `stretch` , ...)
- `gap` om ruimte tussen de flex-items te creëren
- `flex-wrap` om te bepalen of de flex-items moeten wrappen naar een nieuwe regel (`nowrap` , `wrap` , ...)

Directe child-elementen van een flex-container zijn flex-items. Enkele belangrijke eigenschappen voor flex-items zijn:

- `flex-grow` om te bepalen hoe snel een flex-item groeit ten opzichte van de andere flex-items. `0` betekent niet groeien, `1` betekent gelijk groeien met de andere flex-items, `2` betekent dubbel zo snel groeien, ...
- `flex-shrink` om te bepalen hoe snel een flex-item krimpt ten opzichte van de andere flex-items. `0` betekent niet krimpen, `1` betekent gelijk krimpen met de andere flex-items, `2` betekent dubbel zo snel krimpen, ...
- `flex-basis` om de basisgrootte van een flex-item te bepalen, bv. `auto` , `100px` , `50%` , ...
- `align-self` om de uitlijning van een specifiek flex-item langs de kruis-as te bepalen. Dit overschrijft de `align-items` -waarde van de flex-container (`flex-start` , `center` , `stretch` , ...)
- `order` om de volgorde van een specifiek flex-item te bepalen ten opzichte van de andere flex-items. Lagere waarden komen eerst
- `flex` als shorthand voor `flex-grow` , `flex-shrink` en `flex-basis` , bv. `1 0 auto` , `2 1 100px` , ...

CSS Reset

```
* {  
  box-sizing: border-box;  
  margin: 0;  
  padding: 0;  
}
```

CSS

- `box-sizing: border-box;` zorgt ervoor dat de `width` en `height` van een element inclusief de `padding` en `border` zijn. Dit maakt consistentere layouts mogelijk omdat je op voorhand weet hoe groot een element zal zijn.
- `margin: 0;` en `padding: 0;` verwijder je de standaard marges en padding van alle elementen. Dit biedt je een consistente basis voor verdere opmaak. Je bepaalt zelf wat de witruimte zal zijn.

Media Queries

Via `media queries` pas je CSS toe op basis van bepaalde voorwaarden, zoals schermgrootte, resolutie, oriëntatie, ...

Je gebruikt dit vaak voor `responsive design` en past de layout en opmaak aan verschillende schermgroottes aan (desktop, tablet, mobiel).

```
@media screen and (max-width: 960px) {  
  section {  
    width: 50%;  
    background-color: lightblue;  
  }  
}
```

CSS

CSS Custom Properties (Variabelen)

Je gebruikt CSS Custom Properties (variabelen) om herbruikbare waarden te definiëren, zoals kleuren, afmetingen, marges, paddings, ...

```

:root {
  --background-color: #333;
  --text-color: white;
  --footer-height: 50px;
}

header,
footer {
  width: 100%;
  background-color: var(--background-color);
  color: var(--text-color);
}

main {
  max-width: 1000px;
  margin: 0 auto;
  padding: 1rem;
  display: flex;
  flex-wrap: wrap;
  margin-bottom: calc(var(--footer-height) + 1rem);
}

footer {
  height: var(--footer-height);
}
```

Met `calc()` reken je binnen CSS-eigenschappen, zoals het optellen van de footer-hoogte met een extra marge.

JavaScript

JavaScript is de programmeertaal waarmee je interactiviteit en dynamische functionaliteit op een webpagina bouwt. Je kan elementen manipuleren, **events** afhandelen en data van een server ophalen.

Variabelen

Je gebruikt variabelen om gegevens op te slaan en later opnieuw te gebruiken:

- **let** voor veranderlijke variabelen
- **const** voor constante variabelen

```
let leeftijd = 34;
const PI = 3.14;

leeftijd = 35;
// PI = 3.141592; // Dit zou een fout geven omdat PI een
// constante is
```

js

Conditionele statements

Je gebruikt conditionele statements om beslissingen te maken in de code:

```
const leeftijd = 20;

if (leeftijd >= 18) {
  // Deze code wordt uitgevoerd omdat 20 >= 18
  console.log("Je bent een volwassene.");
} else {
  console.log("Je bent een minderjarige.");
}
```

js

Iteratieve statements

Je gebruikt iteratieve statements om blokken code herhaaldelijk uit te voeren:

Een **gesloten** iteratie, je bepaalt op voorhand hoe vaak de code herhaalt:

```
for (let i = 0; i < 5; i++) {  
  console.log("Dit is iteratie nummer " + i);  
}
```

js

Een **open** iteratie, je herhaalt de code totdat een bepaalde voorwaarde niet meer waar is:

```
let userInput = prompt("Typ 'stop' om te stoppen:");  
  
while (userInput.toLowerCase() !== "stop") {  
  console.log("Je typte: " + userInput);  
  userInput = prompt("Typ 'stop' om te stoppen:");  
}
```

js

Je weet op voorhand niet hoe vaak de code herhaalt, de gebruikersinput bepaalt dat.

Functies

Je gebruikt functies om herbruikbare blokken code te definiëren:

```
function volledigeNaam(voornaam, achternaam) {  
  return voornaam + " " + achternaam;  
}  
  
const naam = volledigeNaam("John", "Doe");  
console.log(naam); // Output: John Doe
```

js

Vaak gebruik je functies in combinatie met `event listeners` om te reageren op gebruikersinteracties, zoals een knopklik of tekstinvoer.

```
const button = document.querySelector("button");

button.addEventListener("click", function () {
  alert("De knop is geklikt!");
});
```

js

Je gebruikt hier ook vaak de `anonieme arrow function` -syntax:

```
button.addEventListener("click", () => {
  alert("De knop is geklikt!");
});
```

js

DOM-manipulatie

De DOM (Document Object Model) is een representatie van de HTML-structuur van een webpagina. Met JavaScript manipuleer je de DOM en voeg je elementen toe, verwijder je ze of wijzig je ze.

```
const nieuwElement = document.createElement("p");
nieuwElement.textContent = "Dit is een nieuw paragraaf-
element.";
document.body.appendChild(nieuwElement);
```

js

Template

Met een `template` -element definieer je HTML-structuur die niet verschijnt, maar die je via JavaScript kan kopiëren om dynamisch elementen toe te voegen aan de pagina.

html

```
<h1>Dit is de titel</h1>

<h2>Hieronder worden de kaarten getoond</h2>
<section></section>

<template id="card-template">
  <article class="card">
    <h3>Kaart Titel</h3>
    <p>
      Dit is een voorbeeld van een kaart. Kaarten worden
      vaak gebruikt om
      inhoud te groeperen in een overzichtelijke lay-out.
    </p>
    <a href="#">Lees meer</a>
  </article>
</template>
```

CSS

```
section {
  display: flex;
  flex-wrap: wrap;
  gap: 1rem;
}

.card {
  width: 200px;
  border: 1px solid #ccc;
  border-radius: 8px;
  padding: 16px;
  box-shadow: 2px 2px 6px rgba(0, 0, 0, 0.1);
  margin: 8px;
}

.card h3 {
  margin-top: 0;
}
```

js

```
const template = document.querySelector("#card-  
template");  
const cardContainer = document.querySelector("section");  
  
for (let i = 1; i <= 5; i++) {  
  const clone = template.content.cloneNode(true);  
  clone.querySelector("h3").textContent = `Kaart Titel  
${i}`;  
  clone.querySelector("p").textContent =  
    `Dit is de inhoud van kaart nummer ${i}.`;   
  clone.querySelector("a").href = `#kaart-${i}`;  
  cardContainer.appendChild(clone);  
}
```

Dit is de titel

Hieronder worden de kaarten getoond

Kaart Titel 1

Dit is de inhoud van kaart
nummer 1.

[Lees meer](#)

Kaart Titel 2

Dit is de inhoud van kaart
nummer 2.

[Lees meer](#)

Kaart Titel 3

Dit is de inhoud van kaart
nummer 3.

[Lees meer](#)

Kaart Titel 4

Dit is de inhoud van kaart
nummer 4.

[Lees meer](#)

Previous page
Overzicht

Next page
Oefeningen